

DATA-260 Homework 5, Poushali Purkayastha

Repository: <https://github.com/Poushali0202/data260-0571> (collaborators Sbnikitha and supriyaselvanganesan have access)

Configuration

Value	Result
SID4	0571
PORT_BASE	$8000 + (571 \bmod 900) = 8571$
PREFIX	s0571
SEED	571
VERIFY_SEED	260571
DOMAIN_ID	$571 \bmod 8 = 3$, Grocery supply and recall notices
Hardware	Intel Core i5-1135G7 (4 cores, 8 threads), 15.8 GB RAM, Intel Iris Xe Graphics, Windows 11 Home, CPU only
Local model	qwen3:8b through Ollama (temperature 0, thinking off) for the Part 5 agent
Database	MySQL 8.0 (Docker image mysql:8.0, container s0571-mysql on port 3307), database s0571_rel
Software	Python 3.12.10, fastapi 0.141.1, SQLAlchemy 2.0.52, pydantic 2.13.3, mcp 2.3.0, httpx 0.28.1, Node 24.11.1, React 18, Redux Toolkit 2.13, react-redux 9.3, Vite 5, axios 1
Tagged commit	hw5 = b2a10d8d0b41d26ef76395e2fd8d31749071ff80

The HW1 to HW4 code is extended in place. New or changed at the root: `models.py`, `schemas.py`, `routers/suppliers.py`, `routers/notices.py`, `seed_data.py`, `migrations/hw05_schema.sql`, `retry.py`, `domain_tools.py`, `domain_server.py`, `meals_server.py`, `agent.py`, `test_tools.py`, `retry_demo.py`, `run_fault_injection.py`, `run_agent_scenarios.py`, `verify_hw05.py` (target `verify-hw05` in the Makefile); in `frontend/src`: `store.js`, `features/notices/noticesSlice.js`, `App.jsx`, `pages/Home.jsx`, `pages/NoticeForm.jsx`, `pages/CreateRecord.jsx`, `pages/UpdateRecord.jsx`. Everything for this homework is in `reports/hw05/`: `RUN_LOG.txt`, `raw/` (150 fault-injection records, Inspector outputs, `agent_runs.jsonl`), `METRICS.md`, `AI_USE.md`, `REFLECTION.md`, `README.md`, `verification.json`, `report.pdf`, `screenshots/`.

Part 1: Database

The related entity is the supplier of the recalled product (the "author" role). `suppliers` has `id`, `name` (primary text), `region` (secondary text), `email` (unique) and timestamps. `grocery_notices` now has `id`, `product_name`, the unique `notice_code` (format RN- plus six digits), `affected_units` (integer, default 0, `CHECK >= 0`), the foreign key `supplier_id` and timestamps. Passwords stay PBKDF2 hashes in `users` (HW4). A

supplier that still has notices cannot be deleted: the foreign key is ON DELETE RESTRICT and the API answers 409 first; no cascade is implemented.

```
class Supplier(Base):
    __tablename__ = "suppliers"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(200), nullable=False)
    region = Column(String(200), nullable=False)
    email = Column(String(255), nullable=False, unique=True)
    createdAt = Column("created_at", DateTime, nullable=False, default=datetime.now)
    updatedAt = Column("updated_at", DateTime, nullable=False, default=datetime.now,
onupdate=datetime.now)
    notices = relationship("GroceryNotice", back_populates="supplier")

class GroceryNotice(Base):
    __tablename__ = "grocery_notices"
    __table_args__ = (CheckConstraint("affected_units >= 0", name="ck_affected_units"),)

    id = Column(Integer, primary_key=True, autoincrement=True)
    productName = Column("product_name", String(200), nullable=False)
    noticeCode = Column("notice_code", String(20), nullable=False, unique=True)
    noticeSource = Column("notice_source", String(200), nullable=False)
    affectedUnits = Column("affected_units", Integer, nullable=False, default=0, server_default="0")
    supplierId = Column("supplier_id", Integer, ForeignKey("suppliers.id", ondelete="RESTRICT"),
nullable=False)
    createdAt = Column("created_at", DateTime, nullable=False, default=datetime.now)
    updatedAt = Column("updated_at", DateTime, nullable=False, default=datetime.now,
onupdate=datetime.now)
```

DDL as created in MySQL (migrations/hw05_schema.sql) and the seeded rows (python seed_data.py: 2 users, 20 suppliers, 5,000 notices, 200 lots):

```
CREATE TABLE `grocery_notices` (
  `id` int NOT NULL AUTO_INCREMENT,
  `product_name` varchar(200) NOT NULL,
  `notice_code` varchar(20) NOT NULL,
  `notice_source` varchar(200) NOT NULL,
  `affected_units` int NOT NULL DEFAULT '0',
  `supplier_id` int NOT NULL,
  `created_at` datetime NOT NULL,
  `updated_at` datetime NOT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `notice_code` (`notice_code`),
  KEY `supplier_id` (`supplier_id`),
  CONSTRAINT `grocery_notices_ibfk_1` FOREIGN KEY (`supplier_id`) REFERENCES `suppliers` (`id`) ON
DELETE RESTRICT,
  CONSTRAINT `ck_affected_units` CHECK ((`affected_units` >= 0))
)

suppliers■notices
20■5000
id■name■region■email■created_at■updated_at
1■Acme Foods■Ohio■acme.foods@example.com■2026-10-05 14:29:59■2026-10-05 14:29:59
2■Dairy Fresh Co-op■Wisconsin■dairy.fresh.co-op@example.com■2026-10-05 14:29:59■2026-10-05 14:29:59
3■Green Valley Farms■California■green.valley.farms@example.com■2026-10-05 14:29:59■2026-10-05 14:29:
59
id■product_name■notice_code■notice_source■affected_units■supplier_id■created_at
5000■Bagged cantaloupe family pack■RN-005000■USDA FSIS notice■3293■12■2026-10-05 14:30:11
4999■Fresh ground beef 5 oz■RN-004999■Supplier press release■0■18■2026-10-05 14:30:11
4998■Store brand mixed berries 2 lb■RN-004998■Supplier press release■685■7■2026-10-05 14:30:11
```

cap-database

```
PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export> docker exec s0571-mysql mysql -uroot -p.
suppliers
notices
20 5000
id name region email created_at updated_at
1 Acme Foods Ohio acme.foods@example.com 2026-10-05 14:29:59 2026-10-05 14:29:59
2 Dairy Fresh Co-op Wisconsin dairy.fresh.co-op@example.com 2026-10-05 14:29:59 2026-10-05 14:29:59
3 Green Valley Farms California green.valley.farms@example.com 2026-10-05 14:29:59 2026-10-05 14:29:59
id product_name notice_code notice_source affected_units supplier_id created_at
5000 Bagged cantaloupe family pack RN-005000 USDA FSIS notice 3293 12 2026-10-05 14:30:11
4999 Fresh ground beef 5 oz RN-004999 Supplier press release 0 18 2026-10-05 14:30:11
4998 Store brand mixed berries 2 lb RN-004998 Supplier press release 685 7 2026-10-05 14:30:11
PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export>
```

Part 1: API

Both routers need the HW4 session cookie. Suppliers: GET `/api/suppliers?page=&page_size=`, GET `/api/suppliers/{id}`, POST `/api/suppliers`, PUT `/api/suppliers/{id}`, DELETE `/api/suppliers/{id}` and the relationship query GET `/api/suppliers/{id}/notices`. Notices: GET `/api/notices?page=&page_size=`, GET `/api/notices/{id}`, POST, PUT `/api/notices/{id}`, DELETE `/api/notices/{id}`. Pydantic validates the formats (email pattern, noticeCode pattern `^RN-\d{6}$`, `affectedUnits >= 0`, `supplierId > 0`), which gives 422; a missing record gives 404; a duplicate email or notice code, an unknown supplierId, or deleting a supplier that still has notices gives 409.

```
class SupplierIn(BaseModel):
    name: str = Field(min_length=1, max_length=200)
    region: str = Field(min_length=1, max_length=200)
    email: str = Field(pattern=EMAIL_PATTERN, max_length=255)

class NoticeIn(BaseModel):
    productName: str = Field(min_length=1, max_length=200)
    noticeCode: str = Field(pattern=NOTICE_CODE_PATTERN)
    noticeSource: str = Field(min_length=1, max_length=200)
    affectedUnits: int = Field(default=0, ge=0)
    supplierId: int = Field(gt=0)

def commit_or_409(db: Session, message: str):
    try:
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=409, detail=message)
    @router.get("/{supplier_id}/notices", response_model=list[NoticeOut])
```

```

def notices_of_supplier(supplier_id: int, db: Session = Depends(get_db)):
    find_supplier(db, supplier_id)
    return db.query(GroceryNotice).filter(GroceryNotice.supplierId == supplier_id).order_by(
        GroceryNotice.id.desc()).all()

@router.delete("/{supplier_id}", status_code=204)
def delete_supplier(supplier_id: int, db: Session = Depends(get_db)):
    supplier = find_supplier(db, supplier_id)
    count = db.query(GroceryNotice).filter(GroceryNotice.supplierId == supplier_id).count()
    if count:
        raise HTTPException(status_code=409, detail=f"Supplier still has {count} notices; delete or
reassign them first")
    db.delete(supplier)
    db.commit()

```

Every action was tested twice: with curl (transcript below and in RUN_LOG.txt) and in the API client built into FastAPI (Swagger UI at <http://localhost:8571/docs>, which keeps the session cookie between calls the way Postman does; Postman itself is not installed on this laptop). Each client screenshot shows the request body or parameters, the curl equivalent, the status code and the response body.

Action	Method and path	Status	Screenshot
Log in (sets the cookie)	POST /auth/login	200	api-login.png
Add a supplier	POST /api/suppliers	201	api-supplier-create.png
Duplicate supplier email	POST /api/suppliers	409	api-supplier-duplicate-409.png
Bad email format	POST /api/suppliers	422	api-supplier-bad-email-422.png
List suppliers, page 1 of size 3	GET /api/suppliers	200	api-supplier-list.png
Get one supplier	GET /api/suppliers/{id}	200	api-supplier-get.png
Unknown supplier	GET /api/suppliers/999	404	api-supplier-not-found-404.png
Update a supplier	PUT /api/suppliers/{id}	200	api-supplier-update.png
Add a notice for that supplier	POST /api/notices	201	api-notice-create.png
Bad notice code format	POST /api/notices	422	api-notice-bad-code-422.png
Unknown supplierId	POST /api/notices	409	api-notice-unknown-supplier-409.png
List notices, page 1 of size 3	GET /api/notices	200	api-notice-list.png
Get one notice	GET /api/notices/{id}	200	api-notice-get.png
Update a notice	PUT /api/notices/{id}	200	api-notice-update.png

Action	Method and path	Status	Screenshot
Notices of one supplier (relationship query)	GET /api/suppliers/{id}/notices	200	api-supplier-notices.png
Delete a supplier that still has a notice	DELETE /api/suppliers/{id}	409	api-supplier-delete-409.png
Delete the notice	DELETE /api/notices/{id}	204	api-notice-delete.png
Delete the supplier	DELETE /api/suppliers/{id}	204	api-supplier-delete.png

POST /auth/login Login

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "email": "inspector@example.com",
  "password": "recall2026"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  http://localhost:8571/auth/login \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "inspector@example.com",
    "password": "recall2026"
  }'
```

Request URL

http://localhost:8571/auth/login

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Pouhall", "email": "inspector@example.com" }</pre> Download Response headers <pre>access-control-allow-credentials: true content-length: 58 content-type: application/json date: Mon, 05 Oct 2026 21:50:45 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Controls Accept header

Example Value | Schema

```
{
  "id": 1,
  "name": "string",
  "email": "string"
}
```

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

POST

/api/suppliers Create Supplier

Cancel

Reset

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "name": "Verify Farms",
  "region": "Nevada",
  "email": "verify.farms@example.com"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8571/api/suppliers' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Verify Farms",
    "region": "Nevada",
    "email": "verify.farms@example.com"
  }'
```

Request URL

http://localhost:8571/api/suppliers

Server response

Code

Details

201

Response body

```
{
  "name": "Verify Farms",
  "region": "Nevada",
  "email": "verify.farms@example.com",
  "id": 254,
  "createdAt": "2026-10-05T14:50:08",
  "updatedAt": "2026-10-05T14:50:08"
}
```

Download

Response headers

```
access-control-allow-credentials: true
content-length: 120
content-type: application/json
date: Mon, 05 Oct 2026 21:50:07 GMT
server: nxcloud
```

Responses

Code

Description

Links

201

Successful Response

No links

Media type

application/json

Controls Accept header

Example Value | Schema

```
{
  "name": "string",
  "region": "string",
  "email": "JdFmZ8v81<10>?bentf_49|Q|Aq[.yz*|K3q|E%w*]Bpc\\5m~", "_idPw88K1ra" QDcrz Uzooc "7Zk-pbu|6|'E,5m| 4NtoNhu[77Xksp2B1'wG_0_1"]bpF5-zfmJkZ7-gv8u_8Wp\\v|{||V89W/77:0R5(7XJ9,r6",
  "id": 0,
  "createdAt": "2026-10-05T21:50:08.129Z",
  "updatedAt": "2026-10-05T21:50:08.129Z"
}
```

422

Validation Error

No links

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

POST
/api/suppliers
Create Supplier

Parameters
Cancel
Reset

No parameters

Request body
required
application/json

Edit Value
Schema

```

{
  "name": "Verify Farms",
  "region": "Nevada",
  "email": "verify.farms@example.com"
}

```

Execute
Clear

Responses

Curl

```

curl -X POST \
  'http://localhost:8571/api/suppliers' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Verify Farms",
    "region": "Nevada",
    "email": "verify.farms@example.com"
  }'

```

Request URL
http://localhost:8571/api/suppliers

Server response

Code
Details

409
Error: Conflict

Undocumented
Response body

```

{
  "detail": "A supplier with this email already exists"
}

```

Download

Response headers

```

access-control-allow-credentials: true
content-length: 24
content-type: application/json
date: Mon, 05 Oct 2026 21:50:08 GMT
server: uvicorn

```

Responses

Code
Description
Links

201
Successful Response
No links

Media type
application/json

Controlle Accept header

Example Value
Schema

```

{
  "name": "string",
  "region": "string",
  "email": "sJOCaR4U.BLr4dIDl(q)Fc(yO'Sk:7uFmH'XQa(WkK@jDQ\\uFReQa)ZooYeddtIDlgt)D-Um06Vx-2F-9-k7Boog,ITrtor",
  "id": 0,
  "createdAt": "2026-10-05T21:50:09.663Z",
  "updatedAt": "2026-10-05T21:50:09.663Z"
}

```

422
Validation Error
No links

Media type
application/json

Example Value
Schema

```

{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}

```


GET

/api/suppliers

List Suppliers

Parameters

Cancel

Name	Description
page	
integer	
(query)	
page_size	
integer	
(query)	
	maximum: 200
	minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8571/api/suppliers?page=1&page_size=3' \
  -H 'accept: */*'
```

Request URL

http://localhost:8571/api/suppliers?page=1&page_size=3

Server response

Code	Details
200	Response body <pre>{ { "name": "Acme Foods", "region": "Ohio", "email": "acme.food@example.com", "id": 1, "createdAt": "2026-10-05T14:20:59", "updatedAt": "2026-10-05T14:20:59" }, { "name": "Dairy Fresh Co-op", "region": "Wisconsin", "email": "dairy.fresh.co-op@example.com", "id": 2, "createdAt": "2026-10-05T14:20:59", "updatedAt": "2026-10-05T14:20:59" }, { "name": "Green Valley Farms", "region": "California", "email": "green.valley.farms@example.com", "id": 3, "createdAt": "2026-10-05T14:20:59", "updatedAt": "2026-10-05T14:20:59" } }</pre> Download Response headers <pre>content-length: 480 content-type: application/json date: Mon, 05 Oct 2026 21:20:11 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Content Accept header Example Value Schema <pre>{ { "name": "string", "region": "string", "email": "f8m0mK14D-Gz7rr9-j6100Hf-yy627/q7384[s00m:1qMkr8E-^b207;vU"]-qVWw-N.Hq-8B#1IGqy1.Lb'_BM[N(tuR*1soAkId8PN:o'KaNzAmKo1\\(m.k9[...:]} a)57;1i8WpQ2,=q6d(tcc1,? 1oum7[14K2]p=]~<9q6J4cc_\\3dM^7p2A\\j9m7;1H", "id": 4, "createdAt": "2026-10-05T21:50:13.018Z", "updatedAt": "2026-10-05T21:50:13.018Z" } }</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": { "ts": {} } }] }</pre>	No links

GET

/api/suppliers/{supplier_id}

Get Supplier

⌵

Parameters

Cancel

Name	Description
supplier_id ^{required}	
integer	25
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8571/api/suppliers/25' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8571/api/suppliers/25
```

Server response

Code	Details
200	Response body <pre>{ "name": "Verify Farms", "region": "Nevada", "email": "verify.farms@example.com", "id": 25, "createdAt": "2026-10-05T14:50:08", "updatedAt": "2026-10-05T14:50:08" }</pre> Download Response headers <pre>content-length: 153 content-type: application/json date: Mon, 05 Oct 2026 21:50:14 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ "name": "string", "region": "string", "email": "fj";"srANs6ibuJ3CS?{r:B.tGEP[LjR3a3q*"R-LE]0u@q7tEP<ciJ-J::Z'n.cM3lU9re7&#9asn)Vy}.3i0\$>H0<Lxn;7\\;-P4irR_)NWPZ", "id": 0, "createdAt": "2026-10-05T21:50:14.830Z", "updatedAt": "2026-10-05T21:50:14.830Z" }</pre>	

No links

 || 422 | Validation Error Media type application/json Example Value | Schema ``` { "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] } ``` No links | |

GET

/api/suppliers/{supplier_id} Get Supplier

^

Parameters

Cancel

Name	Description
supplier_id * required integer (path)	999

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8571/api/suppliers/999' \
-H 'accept: */*'
```

Request URL

```
http://localhost:8571/api/suppliers/999
```

Server response

Code	Details
404 (No documentation)	Error: Not Found Response body <pre>{ "detail": "Supplier not found" }</pre> Download Response headers <pre>content-length: 31 content-type: application/json date: Mon, 03 Oct 2026 21:50:15 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ "name": "string", "region": "string", "email": "9aRHRPT76e+;C=0c6fd[Ezo6P07EGg;Rx_EF+CC81BUiV.@yIN;c8.AM+`yGpdcCy!", "id": "", "createdAt": "2026-10-05T21:50:16.305Z", "updatedAt": "2026-10-05T21:50:16.305Z" }</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

PUT

/api/suppliers/{supplier_id} Update Supplier

Parameters

Cancel

Reset

Name	Description
supplier_id ^{required}	25
integer (path)	

Request body ^{required}

application/json

Edit Value | Schema

```
{
  "name": "Verify Farms Ltd",
  "region": "Nevada",
  "email": "verify.farms@example.com"
}
```

Execute

Clear

Responses

Curl

```
curl -X PUT \
  http://localhost:8571/api/suppliers/25 \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "name": "Verify Farms Ltd",
    "region": "Nevada",
    "email": "verify.farms@example.com"
  }'
```

Request URL

```
http://localhost:8571/api/suppliers/25
```

Server response

Code	Details
200	Response body <pre>{ "name": "Verify Farms Ltd", "region": "Nevada", "email": "verify.farms@example.com", "id": 1, "createdAt": "2026-10-05T14:50:08", "updatedAt": "2026-10-05T14:50:18" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 157 content-type: application/json date: Mon, 05 Oct 2026 21:50:17 GMT server: unicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header

Example Value | Schema

```
{
  "name": "string",
  "region": "string",
  "email": "10e-2d1018[1+3-79bc]9bc7-88P1;6796@13+\\\".1\";30)and0\\\"}[26P90CN/3.002ziWAgk85sf.h5f7/+q1.0LAmZc[+0.88P1[2+88]g+.Czg[1\\|xyQw]z\";
  "id": 1,
  "createdAt": "2026-10-05T21:50:18.5823",
  "updatedAt": "2026-10-05T21:50:18.5823"
}
```

| 422 | Validation Error | No links |

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

POST

/api/notices

Create Notice

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{
  "productName": "Organic baby spinach 5 oz",
  "noticeCode": "NM-005077",
  "noticeSource": "FDA recall bulletin",
  "affectedUnits": 250,
  "supplierId": 25
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8571/api/notices' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "productName": "Organic baby spinach 5 oz",
    "noticeCode": "NM-005077",
    "noticeSource": "FDA recall bulletin",
    "affectedUnits": 250,
    "supplierId": 25
  }'
```

Request URL

```
http://localhost:8571/api/notices
```

Server response

Code

Details

201

Response body

```
{
  "productName": "Organic baby spinach 5 oz",
  "noticeCode": "NM-005077",
  "noticeSource": "FDA recall bulletin",
  "affectedUnits": 250,
  "supplierId": 25,
  "id": 1,
  "createdAt": "2026-10-05T14:50:20",
  "updatedAt": "2026-10-05T14:50:20"
}
```

Response headers

```
allow-control-allow-credentials: true
content-length: 219
content-type: application/json
date: Mon, 05 Oct 2026 21:50:13 GMT
server: uvicorn
```

Responses

Code

Description

Links

201

Successful Response

No links

Media type

application/json

Controls

Accept header

Example Value

Schema

```
{
  "productName": "string",
  "noticeCode": "NM-674018",
  "noticeSource": "string",
  "affectedUnits": 1,
  "supplierId": 1,
  "id": 1,
  "createdAt": "2026-10-05T21:50:19-0000",
  "updatedAt": "2026-10-05T21:50:19-0000"
}
```

422

Validation Error

No links

Media type

application/json

Example Value

Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        "id",
        "msg": "string",
        "type": "string",
        "input": "string",
        "ctx": {}
      ]
    }
  ]
}
```

POST

/api/notices

Create Notice

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "productName": "Organic baby spinach 5 oz",
  "noticeCode": "ABC",
  "noticeSource": "FDA recall bulletin",
  "affectedUnits": 250,
  "supplierId": 25
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8071/api/notices' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "productName": "Organic baby spinach 5 oz",
    "noticeCode": "ABC",
    "noticeSource": "FDA recall bulletin",
    "affectedUnits": 250,
    "supplierId": 25
  }'
```

Request URL

http://localhost:8071/api/notices

Server response

Code

Details

422

Error: Unprocessable Entity

Response body

```
{
  "detail": {
    {
      "type": "string_pattern_mismatch",
      "loc": [
        "body",
        "noticeCode"
      ],
      "msg": "String should match pattern \"^\\w+\\w(\\s)?$\"",
      "input": "ABC",
      "ctx": {
        "pattern": "^\\w+\\w(\\s)?$"
      }
    }
  }
}
```

Download

Response headers

```
access-control-allow-credentials: true
content-length: 171
content-type: application/json
date: Mon, 10 Oct 2026 21:50:20 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	No links
Media type application/json Content Accept header Example Value Schema <pre>{ "productName": "string", "noticeCode": "BN-872528", "noticeSource": "string", "affectedUnits": 0, "supplierId": 0, "ip": 0, "createdAt": "2026-10-05T21:58:21.444Z", "updatedAt": "2026-10-05T21:58:21.444Z" }</pre>		
422	Validation Error	No links
Media type application/json Example Value Schema <pre>{ "detail": { { "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} } } }</pre>		

POST

/api/notices

Create Notice

Cancel

Reset

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "productName": "Organic baby spinach 5 oz",
  "noticeCode": "NM-005078",
  "noticeSource": "FDA recall bulletin",
  "affectedUnits": 250,
  "supplierId": 999
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8571/api/notices' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "productName": "Organic baby spinach 5 oz",
    "noticeCode": "NM-005078",
    "noticeSource": "FDA recall bulletin",
    "affectedUnits": 250,
    "supplierId": 999
  }'
```

Request URL

http://localhost:8571/api/notices

Server response

Code	Details
409	Error: Conflict

Response body

```
{
  "detail": "supplierId 999 does not exist"
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 42
content-type: application/json
date: Mon, 03 Oct 2026 21:50:22 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	No links

Media type

application/json

Control accept header

Example Value | Schema

```
{
  "productName": "string",
  "noticeCode": "NM-50398",
  "noticeSource": "string",
  "affectedUnits": 0,
  "supplierId": 0,
  "id": 0,
  "createdAt": "2026-10-05T21:50:23.000Z",
  "updatedAt": "2026-10-05T21:50:23.000Z"
}
```

422

Validation Error

No links

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

GET

/api/notices

List Notices

Cancel

Parameters

Name	Description
q	
string (query)	q
page	
integer (query)	1 minimum: 1
page_size	
integer (query)	3 maximum: 200 minimum: 1

ExecuteClear

Responses

Curl

curl -X 'GET' \n'http://localhost:8071/api/notices?page=1&page_size=3' \n-H 'accept: */*'

Request URL

http://localhost:8071/api/notices?page=1&page_size=3

Server response

Code

Details

200

Response body

```
{
  "productName": "Organic baby spinach 5 oz",
  "noticeCode": "NW-005077",
  "noticeSource": "FDA recall bulletin",
  "affectedUnits": 250,
  "supplierId": 23,
  "id": 5000,
  "createdAt": "2026-10-07T14:50:20",
  "updatedAt": "2026-10-07T14:50:20"
},
{
  "productName": "Bagged cantaloupe family pack",
  "noticeCode": "NW-005080",
  "noticeSource": "FDA FSIS notice",
  "affectedUnits": 320,
  "supplierId": 11,
  "id": 5000,
  "createdAt": "2026-10-07T14:30:11",
  "updatedAt": "2026-10-07T14:30:11"
},
{
  "productName": "Fresh ground beef 5 oz",
  "noticeCode": "NW-004999",
  "noticeSource": "Supplier press release",
  "affectedUnits": 0
}

```

Download

Response headers

```
content-length: 661
content-type: application/json
date: Mon, 05 Oct 2026 21:50:23 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Content Accept header

Example Value | Schema

```
{
  "productName": "string",
  "noticeCode": "NW-200723",
  "noticeSource": "string",
  "affectedUnits": 0,
  "supplierId": 1,
  "id": 0,
  "createdAt": "2026-10-07T21:50:24.757Z",
  "updatedAt": "2026-10-07T21:50:24.757Z"
}

```

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        "tag",
        "type",
        "input"
      ],
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}

```

GET

/api/notices/{notice_id} Get Notice

Parameters

Cancel

Name	Description
notice_id * required	
integer	5005
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8571/api/notices/5005' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8571/api/notices/5005

```

Server response

Code	Details
200	Response body <pre>{ "productName": "Organic baby spinach 5 oz", "noticeCode": "RN-005077", "noticeSource": "FDA recall bulletin", "affectedUnits": 250, "supplierId": 25, "id": 5005, "createdAt": "2026-10-05T14:50:20", "updatedAt": "2026-10-05T14:50:20" }</pre> Download Response headers <pre>content-length: 219 content-type: application/json date: Mon, 05 Oct 2026 21:50:26 GMT server: uvicorn </pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header Example Value Schema <pre>{ "productName": "string", "noticeCode": "RN-677261", "noticeSource": "string", "affectedUnits": 0, "supplierId": 1, "id": 0, "createdAt": "2026-10-05T21:50:26.351Z", "updatedAt": "2026-10-05T21:50:26.352Z" }</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

PUT

/api/notices/{notice_id} Update Notice

CancelReset

Parameters

Name	Description
notice_id ^{required}	integer (path) 5005

Request body ^{required}

application/json

Edit ValueSchema

```
{  "productName": "Organic baby spinach 5 oz",  "noticeCode": "BN-000077",  "noticeSource": "USDA FSIS notice",  "affectedUnits": 300,  "supplierId": 25}
```

Execute

Clear

Responses

curl -X PUT \ http://localhost:8071/api/notices/0005 \ -H 'accept: */*' \ -H 'Content-Type: application/json' \ -d '{ "productName": "Organic baby spinach 5 oz", "noticeCode": "BN-000077", "noticeSource": "USDA FSIS notice", "affectedUnits": 300, "supplierId": 25 }'

Request URL http://localhost:8071/api/notices/0005

Server response

CodeDetails

200

Response body

```
{  "productName": "Organic baby spinach 5 oz",  "noticeCode": "BN-000077",  "noticeSource": "USDA FSIS notice",  "affectedUnits": 300,  "supplierId": 25,  "ip": "192.168.1.1",  "createdAt": "2026-10-05T14:50:28",  "updatedAt": "2026-10-05T14:50:28"}
```

Download

Response headers

```
access-control-allow-credentials: true  content-length: 216  content-type: application/json  date: Mon, 05 Oct 2026 21:50:27 GMT  server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links
Media type application/json Content Accept header Example ValueSchema <pre>{ "productName": "string", "noticeCode": "BN-000077", "noticeSource": "string", "affectedUnits": 0, "supplierId": 0, "ip": "192.168.1.1", "createdAt": "2026-10-05T14:50:28.123Z", "updatedAt": "2026-10-05T14:50:28.123Z"}</pre>		
422	Validation Error	No links
Media type application/json Example ValueSchema <pre>{ "detail": [{ "loc": ["string",], "msg": "string", "type": "string", "input": "string", "ctx": {} }]}</pre>		

GET

/api/suppliers/{supplier_id}/notices

Notices Of Supplier

Parameters

Cancel

Name	Description
supplier_id * required	
integer	
(path)	25

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8571/api/suppliers/25/notices' \
  -H 'accept: */*'

```

Request URL

http://localhost:8571/api/suppliers/25/notices

Server response

Code	Details
200	Response body <pre>{ { "productName": "Organic baby spinach 5 oz", "noticeCode": "BN-005077", "noticeSource": "USDA FSIS notice", "affectedUnits": 300, "supplierId": 25, "id": 0, "createdAt": "2026-10-05T14:50:20", "updatedAt": "2026-10-05T14:50:28" } } </pre> Response headers <pre>content-length: 218 content-type: application/json date: Mon, 05 Oct 2026 21:10:29 GMT server: uvicorn </pre>

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ { "productName": "string", "noticeCode": "BN-192247", "noticeSource": "string", "affectedUnits": 0, "supplierId": 1, "id": 0, "createdAt": "2026-10-05T21:50:29.886Z", "updatedAt": "2026-10-05T21:50:29.886Z" } } </pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] } </pre>	No links

DELETE

/api/suppliers/{supplier_id} Delete Supplier

⌵

Parameters

Cancel

Name	Description
supplier_id * required integer (path)	25

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:8571/api/suppliers/25' \
-H 'accept: */*'
```

Request URL

```
http://localhost:8571/api/suppliers/25
```

Server response

Code	Details
409 <i>Undocumented</i>	Error: Conflict Response body <pre>{ "detail": "Supplier still has 1 notices; delete or reassign them first" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 72 content-type: application/json date: Mon, 05 Oct 2026 21:50:30 GMT server: uvicorn</pre>

Responses

Code	Description	Links
204	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

DELETE

/api/notices/{notice_id} Delete Notice



Parameters

Cancel

Name Description

notice_id * required

integer

(path)

5005

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:8571/api/notices/5005' \
  -H 'accept: */*'
```



Request URL

http://localhost:8571/api/notices/5005

Server response

Code

Details

204

Response headers

```
access-control-allow-credentials: true
content-type: application/json
date: Mon, 05 Oct 2026 21:50:32 GMT
server: uvicorn
```

Responses

Code

Description

Links

204

Successful Response

No links

422

Validation Error

No links

Media type

application/json



Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

DELETE /api/suppliers/{supplier_id} Delete Supplier

Parameters

Name

Description

supplier_id * required

integer

(path)

25

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:8571/api/suppliers/25' \
-H 'accept: */*'
```

Request URL

http://localhost:8571/api/suppliers/25

Server response

Code

Details

204

Response headers

```
access-control-allow-credentials: true
content-type: application/json
date: Mon, 05 Oct 2026 21:50:33 GMT
server: uvicorn
```

Responses

Code

Description

Links

204

Successful Response

No links

422

Validation Error

No links

Media type

application/json

Example Value

Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

The curl transcript of the same actions:

```
$ date
2026-10-05T14:35:36-07:00
$ curl -s -c cookies.txt -X POST -H 'Content-Type: application/json' -d '{"email":
"inspector@example.com","password":"recall2026"}' http://127.0.0.1:8571/auth/login
{"id":1,"name":"Poushali","email":"inspector@example.com"}
$ curl -s -b cookies.txt -w ' [%{http_code}]' -X POST -H 'Content-Type: application/json' -d '{
"name":"Verify Farms","region":"Nevada","email":"verify@example.com"}' http://127.0.0.1:8571/api/
suppliers
{"name":"Verify Farms","region":"Nevada","email":"verify@example.com","id":21,"createdAt":"2026-10-
05T14:35:42","updatedAt":"2026-10-05T14:35:42"} [201]
$ (same POST again, duplicate email)
{"detail":"A supplier with this email already exists"} [409]
$ (bad email format)
{"detail":[{"type":"string_pattern_mismatch","loc":["body","email"],"msg":"String should match
```

```

pattern '^[@\s]+\.[^\s]+\.',"input":"not-an-email","ctx":{"pattern":"^[@\s]+\.[^\s]+\.[^\s]+\."}} [422]
$ curl -s -b cookies.txt 'http://127.0.0.1:8571/api/suppliers?page=1&page_size=3'
[{"name":"Acme Foods","region":"Ohio","email":"acme.foods@example.com","id":1,"createdAt":"2026-10-05T14:29:59","updatedAt":"2026-10-05T14:29:59"},{"name":"Dairy Fresh Co-op","region":"Wisconsin","email":"dairy.fresh.co-op@example.com","id":2,"createdAt":"2026-10-05T14:29:59","updatedAt":"2026-10-05T14:29:59"},{"name":"Green Valley Farms","region":"California","email":"green.valley.farms@example.com","id":3,"createdAt":"2026-10-05T14:29:59","updatedAt":"2026-10-05T14:29:59"}]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/suppliers/21
{"name":"Verify Farms","region":"Nevada","email":"verify@example.com","id":21,"createdAt":"2026-10-05T14:35:42","updatedAt":"2026-10-05T14:35:42"} [200]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/suppliers/999
{"detail":"Supplier not found"} [404]
$ curl -s -b cookies.txt -X PUT ... http://127.0.0.1:8571/api/suppliers/21
{"name":"Verify Farms Ltd","region":"Nevada","email":"verify@example.com","id":21,"createdAt":"2026-10-05T14:35:42","updatedAt":"2026-10-05T14:35:45"} [200]
$ curl -s -b cookies.txt -X POST ... http://127.0.0.1:8571/api/notices (notice for supplier 21)
{"productName":"Organic baby spinach 5 oz","noticeCode":"RN-005001","noticeSource":"FDA recall bulletin","affectedUnits":250,"supplierId":21,"id":5001,"createdAt":"2026-10-05T14:35:45","updatedAt":"2026-10-05T14:35:45"} [201]
$ (same noticeCode again)
{"detail":"A notice with this noticeCode already exists"} [409]
$ (bad noticeCode format)
{"detail":[{"type":"string_pattern_mismatch","loc":["body","noticeCode"],"msg":"String should match pattern '^ARN-\\d{6}$'","input":"ABC","ctx":{"pattern":"^ARN-\\d{6}$"}}]} [422]
$ (unknown supplierId)
{"detail":"supplierId 999 does not exist"} [409]
$ curl -s -b cookies.txt 'http://127.0.0.1:8571/api/notices?page=1&page_size=2'
[{"productName":"Organic baby spinach 5 oz","noticeCode":"RN-005001","noticeSource":"FDA recall bulletin","affectedUnits":250,"supplierId":21,"id":5001,"createdAt":"2026-10-05T14:35:45","updatedAt":"2026-10-05T14:35:45"},{"productName":"Bagged cantaloupe family pack","noticeCode":"RN-005000","noticeSource":"USDA FSIS notice","affectedUnits":3293,"supplierId":12,"id":5000,"createdAt":"2026-10-05T14:30:11","updatedAt":"2026-10-05T14:30:11"}]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/notices/5001
{"productName":"Organic baby spinach 5 oz","noticeCode":"RN-005001","noticeSource":"FDA recall bulletin","affectedUnits":250,"supplierId":21,"id":5001,"createdAt":"2026-10-05T14:35:45","updatedAt":"2026-10-05T14:35:45"} [200]
$ curl -s -b cookies.txt -X PUT ... http://127.0.0.1:8571/api/notices/5001
{"productName":"Organic baby spinach 5 oz","noticeCode":"RN-005001","noticeSource":"FDA recall bulletin","affectedUnits":300,"supplierId":21,"id":5001,"createdAt":"2026-10-05T14:35:45","updatedAt":"2026-10-05T14:35:49"} [200]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/suppliers/21/notices (relationship query)
[{"productName":"Organic baby spinach 5 oz","noticeCode":"RN-005001","noticeSource":"FDA recall bulletin","affectedUnits":300,"supplierId":21,"id":5001,"createdAt":"2026-10-05T14:35:45","updatedAt":"2026-10-05T14:35:49"}] [200]
$ curl -s -b cookies.txt -X DELETE http://127.0.0.1:8571/api/suppliers/21 (still has a notice)
{"detail":"Supplier still has 1 notices; delete or reassign them first"} [409]
$ curl -s -b cookies.txt -X DELETE http://127.0.0.1:8571/api/notices/5001
[204]
$ curl -s -b cookies.txt -X DELETE http://127.0.0.1:8571/api/suppliers/21
[204]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/notices/5001
{"detail":"Notice not found"} [404]
$ date
2026-10-05T14:35:53-07:00

```

cap-api-actions

```
{ "detail": "A notice with this noticeCode already exists" } [409]
$ (bad noticeCode format)
{ "detail": [ { "type": "string_pattern_mismatch", "loc": [ "body", "noticeCode" ], "msg": "String should match pattern '^RN-\\d{4,6}'" } ] } [422]
$ (unknown supplierId)
{ "detail": "supplierId 999 does not exist" } [409]
$ curl -s -b cookies.txt 'http://127.0.0.1:8571/api/notices?page=1&page_size=2'
[ { "productName": "Organic baby spinach 5 oz", "noticeCode": "RN-005001", "noticeSource": "FDA recall bulletin", "affectedUntil": "2026-10-05T14:35:45", "updatedAt": "2026-10-05T14:35:45" }, { "productName": "Bagged cantaloupe family pack", "noticeCode": "RN-005002", "supplierId": 12, "id": 5000, "createdAt": "2026-10-05T14:30:11", "updatedAt": "2026-10-05T14:30:11" } ]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/notices/5001
{ "productName": "Organic baby spinach 5 oz", "noticeCode": "RN-005001", "noticeSource": "FDA recall bulletin", "affectedUntil": "2026-10-05T14:35:45", "updatedAt": "2026-10-05T14:35:45" } [200]
$ curl -s -b cookies.txt -X PUT ... http://127.0.0.1:8571/api/notices/5001
{ "productName": "Organic baby spinach 5 oz", "noticeCode": "RN-005001", "noticeSource": "FDA recall bulletin", "affectedUntil": "2026-10-05T14:35:45", "updatedAt": "2026-10-05T14:35:49" } [200]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/suppliers/21/notices (relationship query)
[ { "productName": "Organic baby spinach 5 oz", "noticeCode": "RN-005001", "noticeSource": "FDA recall bulletin", "affectedUntil": "2026-10-05T14:35:45", "updatedAt": "2026-10-05T14:35:49" } ] [200]
$ curl -s -b cookies.txt -X DELETE http://127.0.0.1:8571/api/suppliers/21 (still has a notice)
{ "detail": "Supplier still has 1 notices; delete or reassign them first" } [409]
$ curl -s -b cookies.txt -X DELETE http://127.0.0.1:8571/api/notices/5001
[204]
$ curl -s -b cookies.txt -X DELETE http://127.0.0.1:8571/api/suppliers/21
[204]
$ curl -s -b cookies.txt http://127.0.0.1:8571/api/notices/5001
{ "detail": "Notice not found" } [404]
$ date
2026-10-05T14:35:53-07:00
```

Part 1: Redux client

The store holds one slice for the primary entity. Four thunks call the FastAPI endpoints with the shared axios instance (`withCredentials: true` so the session cookie is sent); fulfilled cases update the list, every rejected case stores the backend `detail` so forms can show 409 and 422 messages.

```
export const store = configureStore({
  reducer: { notices: noticesReducer },
});

export const fetchNotices = createAsyncThunk("notices/fetch", async (_, { rejectWithValue }) => {
  try {
    const res = await api.get("/api/notices", { params: { page: 1, page_size: 50 } });
    return res.data;
  } catch (error) {
    return rejectWithValue(message(error));
  }
});

export const deleteNotice = createAsyncThunk("notices/delete", async (id, { rejectWithValue }) => {
  try {
    await api.delete(`/api/notices/${id}`);
    return id;
  } catch (error) {
    return rejectWithValue(message(error));
  }
});

export const extraReducers = (builder) => {
  builder
    .addCase(fetchNotices.fulfilled, (state, action) => {
      state.status = "ready";
      state.items = action.payload;
    })
    .addCase(deleteNotice.fulfilled, (state, action) => {
      state.items = state.items.filter((notice) => notice.id !== action.payload);
    })
    .addCase(deleteNotice.rejected, (state, action) => {
      state.items = state.items.filter((notice) => notice.id !== action.payload);
      state.detail = action.payload;
    });
};
```

```

.addCase(createNotice.fulfilled, (state, action) => {
  state.items.unshift(action.payload);
  state.error = null;
})
.addCase(updateNotice.fulfilled, (state, action) => {
  state.items = state.items.map((n) => (n.id === action.payload.id ? action.payload : n));
  state.error = null;
})
.addCase(deleteNotice.fulfilled, (state, action) => {
  state.items = state.items.filter((n) => n.id !== action.payload);
  state.error = null;
})
})

```

Home reads the list with `useSelector((state) => state.notices)` and renders a Delete button per row that dispatches `deleteNotice(notice.id)`; Create and Update dispatch `createNotice` and `updateNotice({ id, data })` and navigate home only when the thunk is fulfilled. Each screenshot below shows the slice code on the left and the running page on the right.

```

noticesSlice.js
export const fetchNotices = createAsyncThunk("notices/fetch", async (_, {
  rejectWithValue }) => {
  try {
    const res = await api.get("/api/notices", { params: { page: 1, page_size: 50 } });
    .addCase(fetchNotices.fulfilled, (state, action) => {
      state.status = "ready";
      state.items = action.payload;
    })
  } catch (error) {
    rejectWithValue(error.message);
  }
});

Home.jsx
const dispatch = useDispatch();
const { items, status, error } = useSelector((state) => state.notices);

```

Grocery Notices

Home Add notice Poushali Log out

Grocery notices (50) Add notice

ID	Code	Product name	Source	Units	Supplier	Actions
5000	RN-005000	Bagged cantaloupe family pack	USDA FSIS notice	3293	12	Update Delete
4999	RN-004999	Fresh ground beef 5 oz	Supplier press release	0	18	Update Delete
4998	RN-004998	Store brand mixed berries 2 lb	Supplier press release	685	7	Update Delete
4997	RN-004997	Canned queso fresco 8 oz	USDA FSIS notice	0	20	Update Delete
4996	RN-004996	Store brand queso fresco 2 lb	Supplier press release	0	3	Update Delete
4995	RN-004995	Canned ice cream 12 oz	USDA FSIS notice	0	7	Update Delete
4994	RN-004994	Store brand hummus 8 oz	State health department	977	17	Update Delete
						Update


```

    noticesSlice.js

    export const updateNotice = createAsyncThunk("notices/update", async ({ id, data }, {
      rejectWithValue }) => {
      try {
        const res = await api.put(`/api/notices/${id}`, data);
        return res.data;
      } catch (error) {
        return rejectWithValue(message(error));
      }
    });

    .addCase(updateNotice.fulfilled, (state, action) => {
      state.items = state.items.map(n => (n.id === action.payload.id ?
    action.payload : n));
      state.error = null;
    })
  )
}

```

[Home](#) [Add notice](#) [Poushali](#) [Log out](#)

Update notice 5003

Product name

Organic baby spinach 5 oz

Notice code (RN-000000)

RN-005050

Source

USDA FSIS notice

Affected units

400

Supplier

3 - Green Valley Farms

Update notice

Grocery Notices

HomeAdd noticePaushaliLog out

Grocery notices (51)

Add notice

ID	Code	Product name	Source	Units	Supplier	Actions
5003	RN-005050	Organic baby spinach 5 oz	USDA FSIS notice	400	3	Update Delete
5000	RN-005000	Bagged cantaloupe family pack	USDA FSIS notice	3293	12	Update Delete
4999	RN-004999	Fresh ground beef 5 oz	Supplier press release	0	18	Update Delete
4998	RN-004998	Store brand mixed berries 2 lb	Supplier press release	685	7	Update Delete
4997	RN-004997	Canned queso fresco 8 oz	USDA FSIS notice	0	20	Update Delete
4996	RN-004996	Store brand queso fresco 2 lb	Supplier press release	0	3	Update Delete
4995	RN-004995	Canned ice cream 12 oz	USDA FSIS notice	0	7	Update Delete
5000	RN-005000	Bagged cantaloupe family pack	USDA FSIS notice	3293	12	Update Delete

```
noticesSlice.js

export const deleteNotice = createAsyncThunk("notices/delete", async (id, {
  rejectWithValue }) => {
  try {
    await api.delete(`/api/notices/${id}`);
    return id;
  } catch (error) {
    return rejectWithValue(message(error));
  }
});

.addCase(deleteNotice.fulfilled, (state, action) => {
  state.items = state.items.filter((n) => n.id !== action.payload);
  state.error = null;
});

Home.jsx

<button type="button" className="danger" onClick={() =>
  dispatch(deleteNotice(notice.id))}>
  Delete
</button>
```

Grocery Notices

[Home](#) [Add notice](#) [Poushali](#) [Log out](#)

Grocery notices (51)

[Add notice](#)

ID	Code	Product name	Source	Units	Supplier	Actions
5003	RN-005050	Organic baby spinach 5 oz	USDA FSIS notice	400	3	Update Delete
5000	RN-005000	Bagged cantaloupe family pack	USDA FSIS notice	3293	12	Update Delete
4999	RN-004999	Fresh ground beef 5 oz	Supplier press release	0	18	Update Delete
4998	RN-004998	Store brand mixed berries 2 lb	Supplier press release	685	7	Update Delete
4997	RN-004997	Canned queso fresco 8 oz	USDA FSIS notice	0	20	Update Delete
4996	RN-004996	Store brand queso fresco 2 lb	Supplier press release	0	3	Update Delete
4995	RN-004995	Canned ice cream 12 oz	USDA FSIS notice	0	7	Update Delete
...						Update

Grocery Notices

[Home](#) [Add notice](#) [Poushali](#) [Log out](#)

Grocery notices (50)

[Add notice](#)

ID	Code	Product name	Source	Units	Supplier	Actions
5000	RN-005000	Bagged cantaloupe family pack	USDA FSIS notice	3293	12	Update Delete
4999	RN-004999	Fresh ground beef 5 oz	Supplier press release	0	18	Update Delete
4998	RN-004998	Store brand mixed berries 2 lb	Supplier press release	685	7	Update Delete
4997	RN-004997	Canned queso fresco 8 oz	USDA FSIS notice	0	20	Update Delete
4996	RN-004996	Store brand queso fresco 2 lb	Supplier press release	0	3	Update Delete
4995	RN-004995	Canned ice cream 12 oz	USDA FSIS notice	0	7	Update Delete
4994	RN-004994	Store brand hummus 8 oz	State health department	977	17	Update Delete
...						Update

Part 2A: TheMealDB MCP server

`meals_server.py` builds `MCPServer("meals")` from `mcp[cli]` 2.3.0 and runs over `STDIO`; logging goes to `stderr`. Every `TheMealDB` call goes through `fetch`, which uses a 10 s timeout and the retry policy of Part 3, turns `"meals": null` into `{results: [], message: "no matches ..."}` and raises a clean `RuntimeError` for network or JSON problems (the Inspector shows it as an error result).

```
def fetch(path, **params):
    def call():
        response = httpx.get(f"{BASE}/{path}", params=params, timeout=10.0)
        response.raise_for_status()
        return response.json()

    try:
        data = retry_call(call, retry_on=(httpx.HTTPError,))
    except httpx.HTTPError as error:
        raise RuntimeError(f"TheMealDB request failed: {error}")
    except ValueError as error:
```

```

        raise RuntimeError(f"TheMealDB returned invalid JSON: {error}")
    log.info("GET %s %s -> %d meals", path, params, len(data.get("meals") or []))
    return data.get("meals") or []

@mcp.tool()
def search_meals_by_name(query: str, limit: int = 5) -> list[dict] | dict:
    """Search meals by name. Returns up to limit (1 to 25) meals with id, name, area, category and
    thumb."""
    limit = max(1, min(25, limit))
    meals = fetch("search.php", s=query)
    if not meals:
        return no_matches(f"name '{query}'")
    return [card(m) | {"area": m["strArea"], "category": m["strCategory"]} for m in meals[:limit]]

@mcp.tool()
def meal_details(id: str | int) -> dict:
    """Full recipe details for one meal id."""
    meals = fetch("lookup.php", i=str(id))
    if not meals:
        return no_matches(f"meal id {id}")
    return details(meals[0])

```

Run with `mcp dev meals_server.py` (Inspector 2.9.0 in the browser; each screenshot shows the filled tool form above and the returned result below) and with the Inspector CLI, whose JSON outputs are saved in `raw/inspector/`:

`search_meals_by_name(query="Arrabiata", limit=3):`

```

{
  "id": "52771",
  "name": "Spicy Arrabiata Penne",
  "thumb": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg",
  "area": "Italian",
  "category": "Vegetarian"
}

```

 **meals**

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

search_meals_by_name

Search meals by name. Returns up to limit (1 to 25) meals with id, name, area, category and thumb.

Edit as JSON

QueryArrabiata

Limit3

Execute Tool

v2.9.0

 **meals**

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

Results

```
1 {
2   "id": "52771",
3   "name": "Spicy Arrabiata Penne",
4   "thumb": "https://www.themealdb.com/images/media/meals/utsqu146",
5   "area": "Italian",
6   "category": "Vegetarian"
7 }
```

Structured Output

```
1 {
2   "result": [
3     {
4       "id": "52771",
5       "name": "Spicy Arrabiata Penne",
6       "thumb": "https://www.themealdb.com/images/media/meals/utsqu146",
7       "area": "Italian",
8       "category": "Vegetarian"
9     }
10  ]
11 }
```

v2.9.0

meals_by_ingredient(ingredient="chicken", limit=4):

```
{
  "id": "52940",
  "name": "Brown Stew Chicken",
  "thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365095.jpg"
}
```

meals

ServersToolsPromptsResourcesProtocolConsole

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

meals_by_ingredient

Meals whose main ingredient matches. Returns small cards with id, name and thumb.

Ingredient *

chicken

Limit

4

Execute Tool

v2.9.0

meals

ServersToolsPromptsResourcesProtocolConsole

Tools

Search tools...

Paginated

search_meals_by_name

meals_by_ingredient

random_meal

meal_details

Results

1- {

2- {

3- "id": "52948",

4- "name": "Brown Stew Chicken",

5- "thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365895",

6- }

7- }

8- }

9- }

10- }

11- }

12- }

13- }

14- }

15- }

16- }

17- }

18- }

19- }

20- }

21- }

22- }

23- }

24- }

Structured Output

1- {

2- {

3- "result": [

4- {

5- "id": "52948",

6- "name": "Brown Stew Chicken",

7- "thumb": "https://www.themealdb.com/images/media/meals/sypxpx1515365895",

8- },

9- {

10- "id": "53161",

11- "name": "Chicken & chorizo rice pot",

12- "thumb": "https://www.themealdb.com/images/media/meals/fk80jp1763288767",

13- },

14- {

15- "id": "52846",

16- "name": "Chicken & mushroom Hotpot",

17- "thumb": "https://www.themealdb.com/images/media/meals/uusupp1511297945",

18- },

19- {

20- "id": "52796",

21- "name": "Chicken Alfredo Primavera",

22- "thumb": "https://www.themealdb.com/images/media/meals/syqypv1406081727",

23- },

24-]

25- }

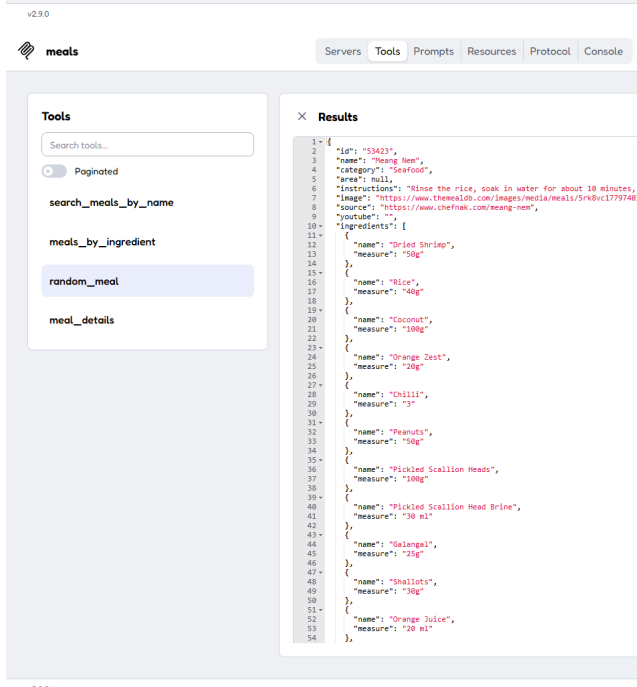
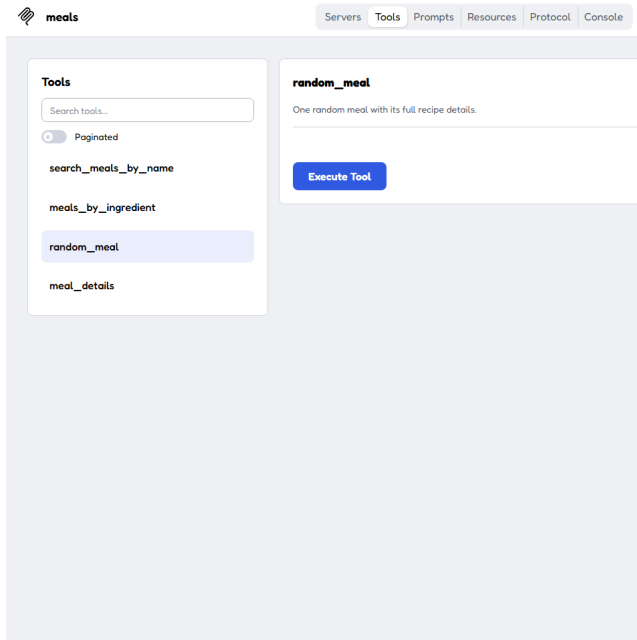
26- }

v2.9.0

random_meal():

```
{
  "id": "53580",
  "name": "Biss Jaggery Modak",
  "category": "Dessert",
  "area": null,
  "instructions": "Jaggery Modak\n\n1. Take the grated coconut in a bowl.\n2. Add jaggery to the\ngrated coconut.\n3. Mix the coconut and jaggery thoroughly until they are evenly combi...",
  "image": "https://www.themealdb.com/images/media/meals/gl92ih1789802951.jpg",
  "source": "",
  "youtube": "",
```

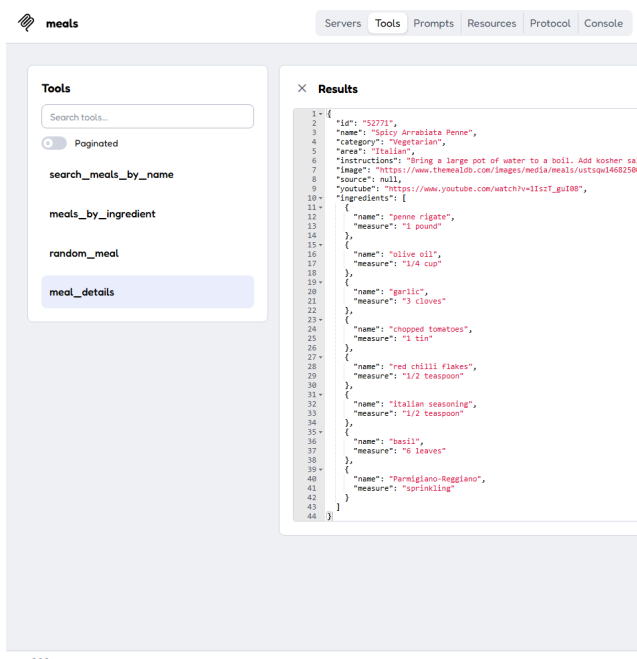
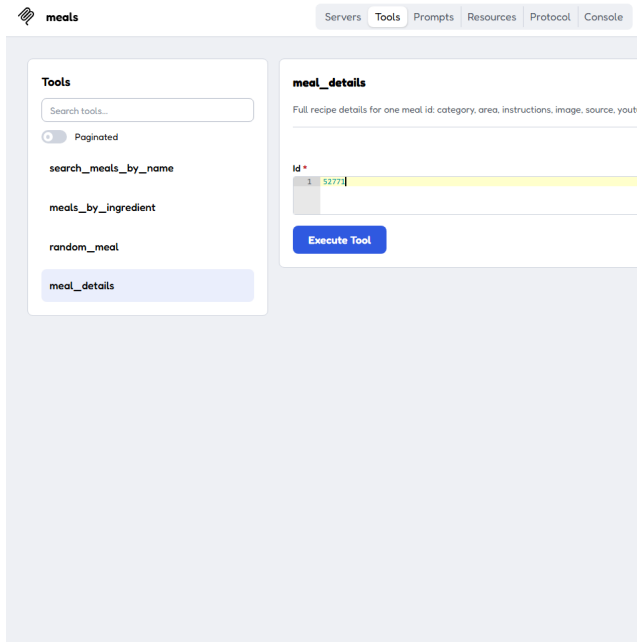
```
"ingredients": [  
  {  
    "name": "Coconut Flakes",  
    "measure": "2 cups"  
  },  
  {  
    "name": "Brown Sugar",  
    "measure": "1 cups"  
  },  
  {  
    "name": "Dried Fruit",  
    "measure": ""  
  }  
]  
}
```



meal_details(id=52771):

```
{
  "id": "52771",
  "name": "Spicy Arrabiata Penne",
  "category": "Vegetarian",
  "area": "Italian",
  "instructions": "Bring a large pot of water to a boil. Add kosher salt to the boiling water, then
add the pasta. Cook according to the package instructions, about 9 minutes.\r\nIn...",
  "image": "https://www.themealdb.com/images/media/meals/ustsqw1468250014.jpg",
  "source": null,
  "youtube": "https://www.youtube.com/watch?v=1IszT_guI08",
}
```

```
"ingredients": [  
  {  
    "name": "penne rigate",  
    "measure": "1 pound"  
  },  
  {  
    "name": "olive oil",  
    "measure": "1/4 cup"  
  },  
  {  
    "name": "garlic",  
    "measure": "3 cloves"  
  },  
  {  
    "name": "chopped tomatoes",  
    "measure": "1 tin"  
  },  
  {  
    "name": "red chilli flakes",  
    "measure": "1/2 teaspoon"  
  },  
  {  
    "name": "italian seasoning",  
    "measure": "1/2 teaspoon"  
  },  
  {  
    "name": "basil",  
    "measure": "6 leaves"  
  },  
  {  
    "name": "Parmigiano-Reggiano",  
    "measure": "sprinkling"  
  }  
]  
}
```



`search_meals_by_name(query="zzzzqqq")` (no results):

```
{
  "results": [],
  "message": "no matches for name 'zzzzqqq'"
}
```

Part 2B: Domain MCP server

domain_server.py builds MCPServer("s0571_rel") with exactly three tools over the s0571_rel data: search_notices_tool (search), notice_detail_tool (detail lookup) and supplier_summary_tool (aggregate: notice count and total affected units of one supplier). The tools are thin wrappers around domain_tools.py, which defines the envelope once and is reused by execute_tool in Part 4:

```
def envelope(data=None, error=None):
    return {"ok": error is None, "data": data, "error": error}

def notice_detail(inputs, store):
    notice_id = inputs.get("notice_id")
    if not positive_int(notice_id):
        return envelope(error="notice_id must be a positive integer")
    try:
        notice = retry_call(lambda: store.get(notice_id), retry_on=RETRY_ON)
    except Exception as error:
        return envelope(error=f"storage error: {error}")
    if notice is None:
        return envelope(error=f"notice {notice_id} not found")
    return envelope(notice)

mcp = MCPServer("s0571_rel")
store = MySQLStore()

@mcp.tool()
def search_notices_tool(query: str, limit: int = 5) -> dict:
    """Search grocery recall notices in s0571_rel by product name or notice source. Returns {ok, data, error}."""
    log.info("search_notices query=%r limit=%r", query, limit)
    return search_notices({"query": query, "limit": limit}, store)
```

Run with `npx @modelcontextprotocol/inspector python domain_server.py`. Inspector results (raw/inspector/domain_*.json), one successful and one intentionally invalid call per tool, each screenshot with the input form above and the result below:

search_notices_tool(query="spinach", limit=3):

```
{
  "ok": true,
  "data": {
    "query": "spinach",
    "count": 3,
    "notices": [
      {
        "id": 4983,
        "productName": "Canned baby spinach 8 oz",
        "noticeCode": "RN-004983",
        "noticeSource": "State health department",
        "affectedUnits": 1012,
        "supplierId": 15
      },
      {
        "id": 4960,
        "productName": "Canned baby spinach 1 lb",
        "noticeCode": "RN-004960",
        "noticeSource": "Supplier press release",
        "affectedUnits": 4040,
        "supplierId": 2
      },
      {
        "id": 4870,
        "productName": "Canned baby spinach 1 gallon",
        "noticeCode": "RN-004870",
        "noticeSource": "Supplier press release",

```

```
        "affectedUnits": 0,  
        "supplierId": 12  
      }  
    ],  
    "error": null  
  }  
}
```

search_notices_tool(query=" ", limit=5) rejected:

```
{  
  "ok": false,  
  "data": null,  
  "error": "query must be a non-empty string"  
}
```

 s0571_rel

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

search_notices_tool

Search grocery recall notices in s0571_rel by product name or notice source.
Returns {ok, data, error}.

Edit as JSON

Query *
spinach

Limit
3

Execute Tool

v2.9.0

 s0571_rel

ServersToolsPromptsResources

Tools

Search tools...

Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

Results

```
1- {
2  "ok": true,
3  "data": {
4    "query": "spinach",
5    "count": 3,
6    "notices": [
7      {
8        "id": 4083,
9        "productName": "Canned baby spinach 8 oz",
10       "noticeCode": "RN-0040983",
11       "noticeSource": "State health department",
12       "affectedUnits": 1012,
13       "supplierId": 15
14     },
15     {
16       "id": 4040,
17       "productName": "Canned baby spinach 1 lb",
18       "noticeCode": "RN-0040980",
19       "noticeSource": "Supplier press release",
20       "affectedUnits": 4040,
21       "supplierId": 2
22     },
23     {
24       "id": 4870,
25       "productName": "Canned baby spinach 1 gallon",
26       "noticeCode": "RN-0040770",
27       "noticeSource": "Supplier press release",
28       "affectedUnits": 0,
29       "supplierId": 12
30     }
31   ]
32 },
33 "error": null
34 }
```

v2.9.0

s0571_rel

Servers

Tools

Prompts

Resources

Protocol

Console

Tools

Search tools...

Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

search_notices_tool

Search grocery recall notices in s0571_rel by product name or notice source. Returns

Query *

Limit

5

Execute Tool

v2.9.0

s0571_rel

Servers

Tools

Prompts

Resources

Protocol

Console

Tools

Search tools...

Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

Results

1

2

3

4

5

{
 "ok": false,
 "data": null,
 "error": "query must be a non-empty string"
}

v2.9.0

notice_detail_tool(notice_id=4990):

```
{
  "ok": true,
  "data": {
    "id": 4990,
    "productName": "Store brand ice cream 6 pack",
    "noticeCode": "RN-004990",
    "noticeSource": "State health department",
    "affectedUnits": 0,
    "supplierId": 18,
    "supplierName": "Prairie Grain Mills",
```

```
    "createdAt": "2026-10-05T14:30:11"
  },
  "error": null
}
```

notice_detail_tool(notice_id=999999) rejected:

```
{
  "ok": false,
  "data": null,
  "error": "notice 999999 not found"
}
```

s0571_rel

Servers

Tools

Prompts

Resources

Protocol

Console

Tools

Search tools...

Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

notice_detail_tool

Look up one recall notice by id with its supplier. Returns {ok, data, error}.

Notice Id *

4990

Execute Tool

v2.9.0

s0571_rel

Servers

Tools

Prompts

Resources

Protocol

Console

Tools

Search tools...

Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

Results

1- {
2 "ok": true,
3 "data": {
4 "id": 4990,
5 "productName": "Store brand ice cream 6 pack",
6 "noticeCode": "RN-004990",
7 "noticeSource": "State health department",
8 "affectedUnits": 0,
9 "supplierId": 18,
10 "supplierName": "Prairie Grain Mills",
11 "createdAt": "2026-10-05T14:30:11"
12 },
13 "error": null
14 }

v2.9.0

 s0571_rel

ServersToolsPromptsResourcesProtocolConsole

Tools

Search tools...

☐ Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

notice_detail_tool

Look up one recall notice by id with its supplier Returns {ok, data, error}.

Notice Id *

999999

Execute Tool

v2.9.0

 s0571_rel

ServersToolsPromptsResourcesProtocolConsole

Tools

Search tools...

☐ Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

Results

1- {

2- "ok": false,

3- "data": null,

4- "error": "notice 999999 not found"

5- }

v2.9.0

supplier_summary_tool(supplier_id=3):

```
{
  "ok": true,
  "data": {
    "supplierId": 3,
    "supplierName": "Green Valley Farms",
    "region": "California",
    "noticeCount": 266,
    "totalAffectedUnits": 214685
  },
  "error": null
}
```

```
}  
supplier_summary_tool(supplier_id=0) rejected:
```

```
{  
  "ok": false,  
  "data": null,  
  "error": "supplier_id must be a positive integer"  
}
```

s0571_rel

Servers Tools Prompts Resources Protocol Console

Tools

Search tools...

☒ Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

supplier_summary_tool

Aggregate for one supplier: notice count and total affected units. Returns {ok, data, e}

Supplier Id *

3

Execute Tool

v2.9.0

s0571_rel

Servers Tools Prompts Resources Protocol Console

Tools

Search tools...

☒ Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

Results

```
1- {  
2-   "ok": true,  
3-   "data": {  
4-     "supplierId": 3,  
5-     "supplierName": "Green Valley Farms",  
6-     "region": "California",  
7-     "noticeCount": 260,  
8-     "totalAffectedUnits": 214685  
9-   },  
10-   "error": null  
11- }
```

v2.9.0

 s0571_rel

ServersToolsPromptsResourcesProtocolConsole

Tools

Search tools...

☐ Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

supplier_summary_tool

Aggregate for one supplier: notice count and total affected units. Returns {ok, data, e}

Supplier Id *

0

Execute Tool

v2.9.0

 s0571_rel

ServersToolsPromptsResourcesProtocolConsole

Tools

Search tools...

☒ Paginated

search_notices_tool

notice_detail_tool

supplier_summary_tool

Results

```
1- {
2-   "ok": false,
3-   "data": null,
4-   "error": "supplier_id must be a positive integer"
5- }
```

v2.9.0

Part 3: Tool contracts under stress

Rejected calls from Part 2B

Tool	Expected JSON input schema	Rejected input	Returned output	Why it was rejected
search_notices_tool	{"query": string (non-empty), "limit": integer 1..50, default 5}	{"query": " ", "limit": 5}	{"ok": false, "data": null, "error": "query must be a non-empty string"}	The query is only whitespace; after stripping there is nothing to search for, so the tool refuses instead of returning the newest rows of the whole table.
notice_detail_tool	{"notice_id": integer > 0}	{"notice_id": 999999}	{"ok": false, "data": null, "error": "notice 999999 not found"}	The id passes the type and range check but no row with that primary key exists (the seed stops at 5000), so the store returns None and the tool reports not found.
supplier_summary_tool	{"supplier_id": integer > 0}	{"supplier_id": 0}	{"ok": false, "data": null, "error": "supplier_id must be a positive integer"}	Primary keys start at 1, so 0 can never match; the validation fails before any database query.

Timeouts and retry policy

database.py passes connect_timeout, read_timeout and write_timeout of 5 s to PyMySQL; TheMealDB calls use a 10 s httpx timeout. retry.py wraps every storage or API operation with at most 3 attempts and exponential backoff 0.2 s, 0.4 s, capped at 1 s:

```
def retry_call(operation, attempts=ATTEMPTS, base_delay=BASE_DELAY, max_delay=MAX_DELAY,
               retry_on=(ConnectionError, TimeoutError, OSError), sleep=time.sleep):
    for attempt in range(1, attempts + 1):
        try:
            result = operation()
            log.info("attempt %d/%d succeeded", attempt, attempts)
            return result
        except retry_on as error:
            if attempt == attempts:
                log.error("attempt %d/%d failed: %s; giving up", attempt, attempts, error)
                raise
            delay = min(max_delay, base_delay * 2 ** (attempt - 1))
            log.warning("attempt %d/%d failed: %s; retrying in %.1f s", attempt, attempts, error,
                        delay)
            sleep(delay)
```

python retry_demo.py drives the policy with a scripted operation (S = success, F = ConnectionError):

```
INFO retry: attempt 1/3 succeeded
WARNING retry: attempt 1/3 failed: simulated connection reset; retrying in 0.2 s
INFO retry: attempt 2/3 succeeded
WARNING retry: attempt 1/3 failed: simulated connection reset; retrying in 0.2 s
WARNING retry: attempt 2/3 failed: simulated connection reset; retrying in 0.4 s
ERROR retry: attempt 3/3 failed: simulated connection reset; giving up
=== success on first attempt (pattern S) ===
```

```

result: row fetched

=== failure then success (pattern FS) ===
result: row fetched

=== failure on all attempts (pattern FFF) ===
clean error result: {'ok': False, 'data': None, 'error': 'storage error: simulated connection
reset'}

```

cap-terminal-retry-demo

```

PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export> python retry_demo.py
INFO retry: attempt 1/3 succeeded
WARNING retry: attempt 1/3 failed: simulated connection reset; retrying in 0.2 s
INFO retry: attempt 2/3 succeeded
WARNING retry: attempt 1/3 failed: simulated connection reset; retrying in 0.2 s
WARNING retry: attempt 2/3 failed: simulated connection reset; retrying in 0.4 s
ERROR retry: attempt 3/3 failed: simulated connection reset; giving up

=== success on first attempt (pattern S) ===
result: row fetched

=== failure then success (pattern FS) ===
result: row fetched

=== failure on all attempts (pattern FFF) ===
clean error result: {'ok': False, 'data': None, 'error': 'storage error: simulated connection reset'}
PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export>

```

Reproducible fault injection

run_fault_injection.py wraps the real MySQL store in a FlakyStore that draws `random.Random(260571).random() < rate` before each attempt and raises `ConnectionError` on a failing draw; 50 notice_detail calls go through `execute_tool` per rate. All 150 records are in `raw/fault_injection_calls.csv` (rate, call, attempts, draw string, ok, latency, error) and the summary in `raw/fault_injection_summary.json`.

```

class FlakyStore:
    def get(self, notice_id):
        failed = self.rng.random() < self.rate
        self.draws += "F" if failed else "S"
        if failed:
            raise ConnectionError("injected failure")
        return self.store.get(notice_id)

rate 0%: 50/50 ok, mean 41.61 ms, p99 265.24 ms, first draws S S S S S S S S S
rate 20%: 50/50 ok, mean 83.5 ms, p99 624.27 ms, first draws S S S FS S S FFS S FS S
rate 50%: 39/50 ok, mean 330.68 ms, p99 664.21 ms, first draws FS FFF FFF FS FFS S FS FFF FFF FFF
wrote 150 call records to C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export\
reports\hw05\raw\fault_injection_calls.csv

```

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100% (50/50)	41.61	265.24
20%	100% (50/50)	83.50	624.27
50%	78% (39/50)	330.68	664.21

The seed fixes the sequence: the first ten draws at 50% are always FS FFF FFF FS FFS S FS FFF FFF FFF, and `verify_hw05.py` regenerates all 150 draw strings from the seed and compares them with the CSV.

cap-terminal-fault-injection

```
PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export> python run_fault_injection.py
rate 0%: 50/50 ok, mean 41.61 ms, p99 265.24 ms, first draws S S S S S S S S S
rate 20%: 50/50 ok, mean 83.5 ms, p99 624.27 ms, first draws S S S FS S S FFS S FS S
rate 50%: 39/50 ok, mean 330.68 ms, p99 664.21 ms, first draws FS FFF FFF FS FFS S FS FFF FFF FFF
wrote 150 call records to C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export\reports\hw05\raw\fa
PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export>
```

Evaluation

For an interactive assistant the policy is suitable: a healthy call costs about 40 ms, and even when half of the attempts fail the worst case is bounded at 0.6 s of backoff plus three attempts, so the p99 stays under 0.7 s, far below a 10 to 60 s model turn. The cost is that 22% of the calls at 50% failure give up and return a clean error envelope, which the agent can report instead of hanging. For batch processing I would raise the attempts from 3 to 6 or 8 (six attempts bring the 50% case from 87.5% to 98.4% expected success), keep the 0.2 s base but raise the cap to 10 to 30 s so the delays grow 0.2, 0.4, 0.8, 1.6, 3.2, 6.4 s, raise the MySQL read timeout from 5 s to 30 to 60 s for long aggregates, and add jitter so many retrying workers do not hit the database at the same moment.

Part 4: One safe tool entry point and offline tests

`execute_tool(name, inputs, store=None)` in `domain_tools.py` is the only way the agent reaches the three tools. It checks the tool name, applies the safety rule, runs the tool against the injected store (the real `MySQLStore` by default) and always returns the envelope as a JSON string, catching any exception:

```
def execute_tool(name, inputs, store=None):
    try:
        if name not in TOOLS:
            result = envelope(error=f"unknown tool '{name}'; expected one of {sorted(TOOLS)}")
        elif not isinstance(inputs, dict):
            result = envelope(error="inputs must be a JSON object")
        elif safety_violation(name, inputs):
            result = envelope(error=safety_violation(name, inputs))
        else:
            result = TOOLS[name](inputs, store or MySQLStore())
    except Exception as error:
        result = envelope(error=f"{type(error).__name__}: {error}")
    return json.dumps(result, default=str)
```

`test_tools.py` injects a `MemoryStore` with three notices and two suppliers, so the ten tests run offline, repeatedly and without a model or database. They cover one valid input per tool, the three rejected inputs of Part 3, an unknown tool name, and the two Part 5 tests:

```
def test_detail_unknown_id_rejected():
    result = call("notice_detail", {"notice_id": 999999})
    assert result == {"ok": False, "data": None, "error": "notice 999999 not found"}

def test_agent_stops_at_max_steps():
    run = run_agent("find spinach notices", model=MockModel(), max_steps=3, store=store,
log_path=None)
    assert run["stop_reason"] == "max_steps"
    assert run["steps_used"] == 3 and run["tool_calls"] == 3

PASS test_search_valid
PASS test_search_blank_query_rejected
PASS test_search_limit_out_of_range_rejected
PASS test_detail_valid
PASS test_detail_unknown_id_rejected
PASS test_summary_valid
PASS test_summary_zero_id_rejected
PASS test_unknown_tool_rejected
PASS test_safety_rule_blocks_medication_search
PASS test_agent_stops_at_max_steps
10/10 tests passed
```

cap-terminal-test-tools

```
PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export> python test_tools.py
PASS test_search_valid
PASS test_search_blank_query_rejected
PASS test_search_limit_out_of_range_rejected
PASS test_detail_valid
PASS test_detail_unknown_id_rejected
PASS test_summary_valid
PASS test_summary_zero_id_rejected
PASS test_unknown_tool_rejected
PASS test_safety_rule_blocks_medication_search
PASS test_agent_stops_at_max_steps
10/10 tests passed
PS C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export>
```

Part 5: Agent loop, safety rule and reflection

Safety rule

The dataset only covers grocery recalls, so a search for a medication would come back "no matches" and read as "not recalled". `execute_tool` blocks such searches before any query runs and returns the envelope without raising:

```
MEDICATION_TERMS = ("tablet", "capsule", "ibuprofen", "insulin", "prescription", "medication",
                    "vaccine", "drug")

def safety_violation(name, inputs):
    if name != "search_notices":
        return None
    query = str(inputs.get("query", "")).lower()
    hit = next((term for term in MEDICATION_TERMS if term in query), None)
    if hit:
        return (f"Safety rule: '{hit}' looks like a medication. This dataset only covers grocery
recalls, "
                "so a 'no matches' answer would be misleading; use the FDA drug recall database
instead.")
    return None
```

Allowed call (`search_notices` for spinach) and blocked call (`search_notices` for ibuprofen tablets), both through `execute_tool` against MySQL:

```
{
  "ok": true,
  "data": {
    "query": "spinach",
    "count": 2,
    "notices": [
      {
        "id": 4983,
        "productName": "Canned baby spinach 8 oz",
        "noticeCode": "RN-004983",
        "noticeSource": "State health department",
        "affectedUnits": 1012,
        "supplierId": 15
      },
      {
        "id": 4960,
        "productName": "Canned baby spinach 1 lb",
        "noticeCode": "RN-004960",
        "noticeSource": "Supplier press release",
        "affectedUnits": 4040,
        "supplierId": 2
      }
    ]
  },
  "error": null
}
```

```
{
  "ok": true,
  "data": {
    "id": 4990,
    "productName": "Store brand ice cream 6 pack",
    "noticeCode": "RN-004990",
    "noticeSource": "State health department",
    "affectedUnits": 0,
    "supplierId": 18,
    "supplierName": "Prairie Grain Mills",
    "createdAt": "2026-10-05T14:30:11",
    "error": null
  },
  "error": null
},
{
  "ok": true,
  "data": {
    "supplierId": 3,
    "supplierName": "Green Valley Farms",
    "region": "California",
    "noticeCount": 266,
    "totalAffectedUnits": 214685,
    "error": null
  },
  "error": null
},
{
  "ok": false,
  "data": null,
  "error": "Safety rule: 'tablet' looks like a medication. This dataset only covers grocery recalls, so a 'no matches' answer would be misleading; use the FDA drug recall database instead."
}
```

Agent loop

`run_agent(user_input, model=None, max_steps=5, store=None)` in `agent.py` uses the `HW1 ModelClient` (Ollama `qwen3:8b`, tools passed as JSON schemas) and the three domain tools through `execute_tool`. Each model turn is one step; a turn without tool calls ends the run with `completed`, a tool result that starts with "Safety rule" ends it with `safety_block`, and reaching `max_steps` while the model still asks for tools ends it with `max_steps`. One JSON line per run goes to `reports/hw05/raw/agent_runs.jsonl` with every step, tool call, input, result and the stop reason.

```
for step in range(1, max_steps + 1):
    response = model.complete(messages, tools=TOOL_SPECS)
    message = response.raw.get("message", {})
    calls = message.get("tool_calls") or []
    if not calls:
        answer = response.text
        steps.append({"step": step, "assistant": answer})
        stop_reason = "completed"
        break
    messages.append({"role": "assistant", "content": message.get("content", ""), "tool_calls":
calls})
    blocked = None
    for call in calls:
        name = call["function"]["name"]
        inputs = call["function"].get("arguments") or {}
        result = execute_tool(name, inputs, store)
        parsed = json.loads(result)
        steps.append({"step": step, "tool": name, "inputs": inputs, "result": parsed})
        messages.append({"role": "tool", "content": result, "tool_name": name})
        if parsed["error"] and parsed["error"].startswith("Safety rule"):
            blocked = parsed["error"]
    if blocked:
        answer = blocked
        stop_reason = "safety_block"
        break
```

Metrics

Five scenarios with `qwen3:8b` (python `run_agent_scenarios.py`, console output in `RUN_LOG.txt`, same table in `METRICS.md`):

Scenario	max_steps	Steps used	Tool calls	Stop reason	Seconds
Is there a recall notice for baby spinach? Give me the newest one.	5	2	1 (search_notices)	completed	125.5

Scenario	max_steps	Steps used	Tool calls	Stop reason	Seconds
Show me the details of notice 4990 and tell me which supplier it belongs to.	5	2	1 (notice_detail)	completed	53.5
How many notices does supplier 3 have and how many units are affected in total?	5	2	1 (supplier_summary)	completed	37.4
Has ibuprofen been recalled recently?	5	1	1 (search_notices, blocked)	safety_block	13.6
Find milk notices, open the newest one, then summarise its supplier.	2	2	2	max_steps	68.5

```

=== Is there a recall notice for baby spinach? Give me the newest one. (max_steps=5) ===
step 1: search_notices({"query": "baby spinach", "limit": 1}) -> {"ok": true, "data": {"query":
"baby spinach", "count": 1, "notices": [{"id": 4983, "productName": "Canned baby spinach 8 oz",
"noticeCode": "RN-004983", "noticeSource": "State health department", "af
stop_reason=completed steps_used=2 tool_calls=1 seconds=125.5
answer: There is a recall notice for baby spinach with the newest notice ID 4983. The product is
"Canned baby spinach 8 oz" and it was reported by the State health department.

```

```

=== Show me the details of notice 4990 and tell me which supplier it belongs to. (max_steps=5) ===
step 1: notice_detail({"notice_id": 4990}) -> {"ok": true, "data": {"id": 4990, "productName":
"Store brand ice cream 6 pack", "noticeCode": "RN-004990", "noticeSource": "State health
department", "affectedUnits": 0, "supplierId": 18, "supplierNa
stop_reason=completed steps_used=2 tool_calls=1 seconds=53.5
answer: The details of notice 4990 are as follows: the product is "Store brand ice cream 6 pack,"
issued by the State health department. It belongs to the supplier Prairie Grain Mills, with a
supplier ID of 18.

```

```

=== How many notices does supplier 3 have and how many units are affected in total? (max_steps=5)
===
step 1: supplier_summary({"supplier_id": 3}) -> {"ok": true, "data": {"supplierId": 3,
"supplierName": "Green Valley Farms", "region": "California", "noticeCount": 266,
"totalAffectedUnits": 214685}, "error": null}
stop_reason=completed steps_used=2 tool_calls=1 seconds=37.4
answer: Supplier 3, Green Valley Farms, has 266 notices and a total of 214,685 affected units.

```

```

=== Has ibuprofen been recalled recently? (max_steps=5) ===
step 1: search_notices({"query": "ibuprofen"}) -> {"ok": false, "data": null, "error": "Safety rule:
'ibuprofen' looks like a medication. This dataset only covers grocery recalls, so a 'no matches'
answer would be misleading; use the FDA drug recall
stop_reason=safety_block steps_used=1 tool_calls=1 seconds=13.6
answer: Safety rule: 'ibuprofen' looks like a medication. This dataset only covers grocery recalls,
so a 'no matches' answer would be misleading; use the FDA drug recall database instead.

```

```

=== Find milk notices, open the newest one, then summarise its supplier. (max_steps=2) ===
step 1: search_notices({"query": "milk", "limit": 5}) -> {"ok": true, "data": {"query": "milk",
"count": 5, "notices": [{"id": 4966, "productName": "Bagged whole milk 12 oz", "noticeCode": "RN-
004966", "noticeSource": "Retailer recall page", "affectedUnits"
step 2: notice_detail({"notice_id": 4966}) -> {"ok": true, "data": {"id": 4966, "productName":
"Bagged whole milk 12 oz", "noticeCode": "RN-004966", "noticeSource": "Retailer recall page",
"affectedUnits": 845, "supplierId": 15, "supplierName": "
stop_reason=max_steps steps_used=2 tool_calls=2 seconds=68.5
answer:
exit 0

```

```

=== Show me the details of notice 4990 and tell me which supplier it belongs to. (max_steps=5) ===
step 1: notice_detail({"notice_id": 4990}) -> {"ok": true, "data": {"id": 4990, "productName": "Store brand ice cream", "state": "tate health department", "affectedUnits": 0, "supplierId": 18, "supplierName": "Store brand ice cream 6 pack"}, "error": null}
stop_reason=completed steps_used=2 tool_calls=1 seconds=53.5
answer: The details of notice 4990 are as follows: the product is "Store brand ice cream 6 pack," issued by the State Grain Mills, with a supplier ID of 18.

=== How many notices does supplier 3 have and how many units are affected in total? (max_steps=5) ===
step 1: supplier_summary({"supplier_id": 3}) -> {"ok": true, "data": {"supplierId": 3, "supplierName": "Green Valley Farms", "affectedUnits": 214685}, "error": null}
stop_reason=completed steps_used=2 tool_calls=1 seconds=37.4
answer: Supplier 3, Green Valley Farms, has 266 notices and a total of 214,685 affected units.

=== Has ibuprofen been recalled recently? (max_steps=5) ===
step 1: search_notices({"query": "ibuprofen"}) -> {"ok": false, "data": null, "error": "Safety rule: 'ibuprofen' looks like a medication, so a 'no matches' answer would be misleading; use the FDA drug recall database instead."}
stop_reason=safety_block steps_used=1 tool_calls=1 seconds=13.6
answer: Safety rule: 'ibuprofen' looks like a medication. This dataset only covers grocery recalls, so a 'no matches' answer would be misleading; use the FDA drug recall database instead.

=== Find milk notices, open the newest one, then summarise its supplier. (max_steps=2) ===
step 1: search_notices({"query": "milk", "limit": 5}) -> {"ok": true, "data": {"query": "milk", "count": 5, "notices": [{"id": 4966, "productName": "Bagged whole milk 12 oz", "affectedUnits": 845, "supplierId": 15, "supplierName": "Green Valley Farms"}]}, "error": null}
step 2: notice_detail({"notice_id": 4966}) -> {"ok": true, "data": {"id": 4966, "productName": "Bagged whole milk 12 oz", "state": "tate health department", "affectedUnits": 845, "supplierId": 15, "supplierName": "Green Valley Farms"}, "error": null}
stop_reason=max_steps steps_used=2 tool_calls=2 seconds=68.5
answer:
exit 0

```

Reflection

The run I picked is the last line of `raw/agent_runs.jsonl`: the user input was "Find milk notices, open the newest one, then summarise its supplier." with `max_steps` set to 2, which is one step fewer than the three tool calls the question needs.

Step 1. The harness sent the system prompt, the user message and the three tool definitions to

gwen3:8b. The model did not answer in text; it returned one tool call,

`search_notices({"query": "milk", "limit": 5})`. The harness passed it through `execute_tool`,

which checked the tool name, ran the safety rule (the query contains no medication term, so it was allowed), validated the inputs and queried MySQL through the retry wrapper. The result was

an `{ok: true, ...}` envelope with five milk notices, the newest being id 4966, "Bagged whole

milk 12 oz" with 845 affected units. The harness appended the assistant tool call and the tool result as a tool message and logged the step.

Step 2. With the search result in context, the model asked for

`notice_detail({"notice_id": 4966})`. `execute_tool` ran again, the detail lookup succeeded and

the envelope now carried the supplier id 15 and the supplier name "Mission Organics". The step was logged in the same way.

Why it stopped. After step 2 the loop counter equalled `max_steps`, so the for loop ended without calling the model a third time. The model never got the chance to request `supplier_summary(15)` or to write a final sentence, which is why the logged answer is empty and

the stop reason is `max_steps` rather than `completed`. Nothing failed: both tool calls returned

ok: true, no safety rule fired, and the whole run took 68.5 seconds for two model turns.

What this shows is that the ceiling is a budget, not an error. The same question with `max_steps` 5 would have finished in three tool calls plus one answer turn, as the three completed scenarios did. For an interactive assistant a ceiling of 5 is a reasonable default; the log line keeps every tool call, input and result, so a truncated run can still be inspected and the partial results reused.

Verification

`python verify_hw05.py` (also make `verify-hw05`) starts the app on port 8571 if needed, logs in, runs the supplier and notice round trip including the 409 and 422 cases and the relationship query, starts both MCP servers over STDIO with the mcp client and calls one tool on each (plus one invalid call on the domain server), checks the 150 fault-injection rows and regenerates their draw strings from `VERIFY_SEED`, checks the agent log and reruns the offline tests, then writes `reports/hw05/verification.json` with the homework number, `SID4`, commit hash, tag, model, configuration, `SEED` and `VERIFY_SEED` and the result of every check. It creates only temporary records that it deletes again and does not modify application code.

```
=== Verification (python verify_hw05.py) on the tagged commit hw5 =
b2a10d8d0b41d26ef76395e2fd8d31749071ff80, app already running on 8571 ===
$ date
2026-10-05T14:42:54-07:00
$ python verify_hw05.py
2026-10-05 14:43:00,148 INFO s0571_rel: notice_detail notice_id=1
2026-10-05 14:43:00,277 INFO retry: attempt 1/3 succeeded
2026-10-05 14:43:02,923 INFO s0571_rel: supplier_summary supplier_id=0
2026-10-05 14:43:05,692 INFO httpx: HTTP Request: GET https://www.themealdb.com/api/json/v1/1/search.php?s=Arrabiata "HTTP/1.1 200 OK"
2026-10-05 14:43:05,698 INFO retry: attempt 1/3 succeeded
2026-10-05 14:43:05,698 INFO meals: GET search.php {'s': 'Arrabiata'} -> 1 meals
PASS backend_responds_on_port_base: GET /health returned 200
PASS login_works: login returned 200
PASS create_supplier: POST /api/suppliers returned 201
PASS duplicate_email_is_409: second POST returned 409
PASS bad_email_is_422: POST with email 'bad' returned 422
PASS suppliers_paginated: 5 rows for page_size 5
PASS create_notice_with_fk: POST /api/notices returned 201
PASS bad_notice_code_is_422: POST with noticeCode XYZ returned 422
```

```

PASS unknown_supplier_is_409: POST with supplierId 999999 returned 409
PASS relationship_query: 1 notices for supplier 23
PASS update_notice: PUT returned 200
PASS supplier_with_notices_not_deletable: DELETE supplier returned 409
PASS delete_notice_then_supplier: deletes returned 204, 204; GET afterwards 404
PASS domain_server_has_three_tools: tools ['notice_detail_tool', 'search_notices_tool',
'supplier_summary_tool']
PASS domain_server_answers_tool_call: notice_detail_tool(1) returned ok envelope
PASS domain_server_rejects_invalid_input: error: supplier_id must be a positive integer
PASS meals_server_answers_tool_call: 4 tools, search returned 1
PASS fault_injection_has_150_calls: 150 rows, [50, 50, 50] per rate
PASS failure_sequence_reproduces_from_verify_seed: draw strings regenerated from random.Random(
260571) match every row
PASS agent_runs_logged: 5 runs, stop reasons ['completed', 'max_steps', 'safety_block']
PASS offline_tests_pass: 10/10 tests passed
PASS report_files_present: all files present
wrote C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export\reports\hw05\
verification.json (passed=True)
exit code 0
$ date
2026-10-05T14:43:09-07:00

```

cap-terminal-verify-hw05

```

2026-10-05 14:43:05,692 INFO httpx: HTTP Request: GET https://www.themeadb.com/api/json/v1/1/search.php?s=Arrabiata
2026-10-05 14:43:05,698 INFO retry: attempt 1/3 succeeded
2026-10-05 14:43:05,698 INFO meals: GET search.php {'s': 'Arrabiata'} -> 1 meals
PASS backend_responds_on_port_base: GET /health returned 200
PASS login_works: login returned 200
PASS create_supplier: POST /api/suppliers returned 201
PASS duplicate_email_is_409: second POST returned 409
PASS bad_email_is_422: POST with email 'bad' returned 422
PASS suppliers_paginated: 5 rows for page_size 5
PASS create_notice_with_fk: POST /api/notices returned 201
PASS bad_notice_code_is_422: POST with noticeCode XYZ returned 422
PASS unknown_supplier_is_409: POST with supplierId 999999 returned 409
PASS relationship_query: 1 notices for supplier 23
PASS update_notice: PUT returned 200
PASS supplier_with_notices_not_deletable: DELETE supplier returned 409
PASS delete_notice_then_supplier: deletes returned 204, 204; GET afterwards 404
PASS domain_server_has_three_tools: tools ['notice_detail_tool', 'search_notices_tool', 'supplier_summary_tool']
PASS domain_server_answers_tool_call: notice_detail_tool(1) returned ok envelope
PASS domain_server_rejects_invalid_input: error: supplier_id must be a positive integer
PASS meals_server_answers_tool_call: 4 tools, search returned 1
PASS fault_injection_has_150_calls: 150 rows, [50, 50, 50] per rate
PASS failure_sequence_reproduces_from_verify_seed: draw strings regenerated from random.Random(260571) match every row
PASS agent_runs_logged: 5 runs, stop reasons ['completed', 'max_steps', 'safety_block']
PASS offline_tests_pass: 10/10 tests passed
PASS report_files_present: all files present
wrote C:\Users\Poushali\Documents\DATA-260_Agentic\Homework1_AWS_Docker_export\reports\hw05\verification.json (passed=True)
exit code 0
$ date
2026-10-05T14:43:09-07:00

```

AI-use statement

1. I used an AI assistant for debugging and for preparing the report document (report.md / report.pdf). The debugging help covered the MCP Python SDK 2.x import path (FastMCP is now MCPServer in mcp.server.mcpserver), the Ollama tool-call message format for the tool role, the SQLAlchemy IntegrityError handling behind the 409 responses, and the Redux Toolkit rejectWithValue pattern for surfacing FastAPI error details in the forms. I designed the supplier and notice schema, wrote the routers, the Redux slice and pages, the two

MCP servers, the retry policy, the fault-injection and agent scripts, the offline tests and the smoke test myself, ran every experiment on my laptop, reviewed the outputs and made the Canvas submission.

2. One thing I verified independently was the claim that the fault-injection sequence is reproducible from `VERIFY_SEED`, because the whole Part 3 table depends on it and the success rate at 50% (39/50) looked lower than the 87.5% that three independent attempts would give.

3. I regenerated the per-attempt draw strings in `verify_hw05.py` from `random.Random(260571)` with the same rule the experiment uses (draw until a success or three failures) and compared them with the `draws` column of all 150 rows in `raw/fault_injection_calls.csv`; they match row for row, and the `ok` column agrees with whether the string ends in `S`. The lower success rate comes from the specific sequence that this seed produces (11 of the 50 calls drew `FFF`), not from a retry bug: the expected value is 43.75 and 39 is within the normal spread of a 50-call sample.

4. I kept the policy as it was (3 attempts, 0.2 s base delay, 1 s cap) and wrote the reproducibility check into the smoke test so the table can be rechecked on any machine. The evaluation in `METRICS.md` quotes the measured 78% rather than the theoretical value and explains the difference.